

Homework 12: A Word Index

Build a small text index. Every step asks you to pick a collection and justify it, because picking correctly is the skill this module is actually teaching.

The Text

```
private static final String TEXT = ""  
    the quick brown fox jumps over the lazy dog  
    the dog barks and the fox runs  
    a quick fox is a happy fox"";
```

Part One: Counting

Create `WordIndex.java` in the classic form.

Count how many times each word appears, then print the three most frequent.

Requirements:

- Use `merge` for the counting. Do **not** write a `get`, null check, `put` sequence.
- The counts must print in a **predictable** order, and a comment must say which map type you chose and why a `HashMap` would have been the wrong call.
- `the` and `fox` both appear four times. Your top-three must be **deterministic** across runs, so decide how to break the tie and say so in a comment.

Part Two: Indexing

Build an index from each initial letter to the set of words starting with it.

Requirements:

- Initials must come out **alphabetically**, without a separate sorting step
- Each word appears **once** per initial, and the words are **sorted**
- Use `computeIfAbsent`, not a null check

Then use sequenced-collection methods to print the first entry, the last entry, and the keys in reverse.

Finally, build the set of unique words **three ways** using `HashSet`, `LinkedHashSet` and `TreeSet`, print all three, and write a comment on what each one promises. One of them promises nothing at all.

Part Three: Lose An Entry

Write a `MutableTag` class with a single renameable field, correct `equals` and `hashCode`, and a `toString`.

Put one in a `HashMap` as the **key**, then rename it. Print, in this order:

1. `get`, `containsKey` and `size` before the rename
2. The same three after
3. The key set, by iterating
4. `size` after calling `remove(tag)`

Explain It

Write a comment covering, in your own words:

- Why `get` returns null when the entry is demonstrably still there
- Why iteration finds it but lookup does not
- Why `remove` cannot delete it either
- What the only remaining way to get rid of it is

Then do the same with a record as the key, and show a freshly constructed equal key still finding the value.

Acceptance Criteria

- ☐ Runs with `java WordIndex.java`
- ☐ Counting uses `merge`
- ☐ A comment justifies the map type over `HashMap`
- ☐ The top-three output is deterministic, with the tiebreak explained
- ☐ The initial index is alphabetical with sorted, deduplicated values
- ☐ `computeIfAbsent` is used
- ☐ `firstEntry`, `lastEntry` and `reversed` all appear
- ☐ All three set types are printed with a comment on what each promises
- ☐ The mutable-key demo shows `get` null, `containsKey` false, `size` 1
- ☐ Iteration still yields the lost entry, and `remove` fails to delete it
- ☐ The explanation covers all four questions above

Stretch

Recover from the lost entry. Without rebuilding the map, remove the unreachable key using an `Iterator`.

Then write a comment on why `Iterator.remove` can do what `Map.remove` cannot, and connect it back to what Module 12 said about `ConcurrentModificationException` being the same machinery viewed from the other side.

Hint, if the top-three keeps changing

Sorting by count alone leaves equal counts in whatever order they happened to be in. That order can come from a hash table, which means it is not stable. Add a second comparison so ties resolve the same way every time.